

Property Configuration in Entity Framework Core using Fluent API

In this article, I will discuss **Property Configuration in Entity Framework Core using Fluent API** with Examples. Please read our previous article discussing [Entity Configurations in Entity Framework Core using Fluent API](#) with Examples.

Property Configurations in Entity Framework Core using Fluent API

In Entity Framework (EF) Core, Property Configurations allow us to define settings and rules specific to individual properties of an entity. This includes specifying column names, data types, default values, nullability, maximum length, precision and scale, computed columns, value conversions, concurrency tokens, and more. By explicitly configuring properties using the Fluent API, we ensure that database schema accurately reflects application requirements, thereby improving data integrity, consistency, and performance.

Let us proceed and understand how and when to apply property-level configurations using Entity Framework Core Fluent API in a .NET console application. We will cover the following property configurations:

- [Configuring Column Names](#)
- [Configuring Data Types](#)
- [Configuring Default Values](#)
- [Configuring Required and Nullable Properties](#)
- [Configuring Maximum Length](#)
- [Configuring Precision and Scale](#)
- [Configuring Computed Columns](#)
- [Configuring Value Conversions](#)
- [Configuring Concurrency Tokens](#)
- [Configuring Shadow Properties](#)
- [Configuring Value Generation \(Identity\)](#)
- [Ignoring Properties](#)

Configuring Column Names

By default, EF Core maps each property of an entity to a database column with the same name as the property name. However, there are scenarios where we might need to specify a different column name to align with existing database conventions, legacy databases, or specific organizational standards.

Suppose we have an entity called Customer with a property called FirstName, but we want it to be mapped to a column called First_Name in the database. Then we need to configure the same using EF Core Fluent API as follows:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .Property(c => c.FirstName)
        .HasColumnName("First_Name");
}
```

Code Explanation:

- The HasColumnName("First_Name") method specifies that the FirstName property should be mapped to a column named First_Name in the database. This allows for consistency with existing database naming

conventions or requirements.

Configuring Data Types

By default, EF Core automatically selects the appropriate database data type based on the .NET type of the property. However, there are cases where we need to specify a particular data type to match existing database schemas, optimize storage, or meet specific application requirements. For example, if we have a decimal property `Price` in the `Product` entity and we want to ensure it is mapped to a SQL Server column of type `decimal(10,2)`, then we need to do the following Fluent API configuration:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Product>()
        .Property(p => p.Price)
        .HasColumnType("decimal(10, 2)");
}
```

Code Explanation:

- **HasColumnType("decimal(10, 2)")**: Explicitly sets the SQL data type of the `Price` property to decimal with a precision of 10 and a scale of 2.
- **Precision and Scale**: Precision (10) defines the total number of digits, while scale (2) defines the number of digits to the right of the decimal point.

Configuring Default Values

Setting default values for properties ensures that when a new record is inserted without explicitly setting a value for a property, the database assigns the predefined default value. This is useful for properties like status indicators, timestamps, or flags. For example, to set a default order status of `Pending` for the `Order` entity, we can configure the following:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>()
        .Property(o => o.Status)
        .HasDefaultValue(OrderStatus.Pending);
}
```

Code Explanation:

- **HasDefaultValue(OrderStatus.Pending)**: Specifies that the default value for the `Status` property is `Pending`. If `OrderStatus` is an enum, EF Core will store its underlying integer value unless a value converter is specified.

Setting Default Values Using SQL Functions:

We can also set default values using SQL functions. For example, setting a default date:

```
modelBuilder.Entity<Order>()
    .Property(o => o.OrderDate)
    .HasDefaultValueSql("GETUTCDATE()");
```

Configuring Required Properties

By default, EF Core determines whether a property is required (non-nullable) or optional (nullable) based on the CLR type of the property. For example, non-nullable types, such as int, decimal, bool, etc., are treated as required. Nullable types such as int?, decimal?, or reference types like string? are optional. However, if we want to explicitly enforce a property to be required, we can use the `IsRequired()` method. For example, to make the Email property of the Customer entity required, we need to do the following Fluent API Configuration:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .Property(c => c.Email)
        .IsRequired();
}
```

Code Explanations:

- The `IsRequired()` method specifies that the Email property cannot be null in the database.

Configuring Nullable Properties

By default, nullable types are considered optional (nullable). However, we can explicitly configure a property to allow null values using the `IsRequired(false)` method. For example, to make the Description property in the Product entity optional, we need to do the following Fluent API Configuration:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Product>()
        .Property(p => p.Description)
        .IsRequired(false);
}
```

Code Explanations:

- The `IsRequired(false)` method allows the Description property to have null values.

Configuring Maximum Length:

When dealing with string properties, it is often necessary to define the maximum length, which will translate into the appropriate column size in the database and prevent users from exceeding the allowed limit, enforcing data integrity and optimizing storage. For example, we can set a maximum length of 100 characters for the FirstName property in the Customer entity as follows:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .Property(c => c.FirstName)
        .HasMaxLength(100);
}
```

Code Explanation:

- **HasMaxLength(100):** Ensures that the FirstName property cannot exceed 100 characters.
- **Database Translation:** Depending on the database provider, this might translate to VARCHAR(100), NVARCHAR(100), etc.

Configuring Precision and Scale

For decimal properties, we can configure the precision (total number of digits) and scale (number of decimal places) to ensure that numeric data is stored accurately, which is critical for financial calculations, measurements, and other precise data types. For example, for a Price property in the Product entity, we can configure the precision and scale as follows:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Product>()
        .Property(p => p.Price)
        .HasPrecision(10, 2);
}
```

Code Explanation:

- **HasPrecision(10, 2):** Sets the Price property to have a precision of 10 digits and a scale of 2 decimal places, corresponding to a SQL Server column of type decimal(10,2). Higher precision and scale consume more storage space, so, balance based on application requirements.

Configuring Computed Columns

Computed columns are database columns whose values are automatically calculated based on other columns. This ensures data consistency and reduces the need for manual calculations in the application layer. For example, if we want to calculate the TotalPrice in the OrderItem entity based on the Quantity and UnitPrice, then we need to configure the same as follows:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OrderItem>()
        .Property(oi => oi.TotalPrice)
        .HasComputedColumnSql("[Quantity] * [UnitPrice]");
}
```

Code Explanation:

- **HasComputedColumnSql("[Quantity] * [UnitPrice]"):** Defines the TotalPrice column as computed by multiplying the Quantity and UnitPrice columns.
- **Database Responsibility:** The database automatically updates the TotalPrice whenever Quantity or UnitPrice changes.

Configuring Value Conversions

Sometimes, we need to store a property in the database differently from how it is represented in the application. This can be done using Value Conversions. This is useful for transforming data types, encrypting data, or storing enums as strings. For example, if we want to store the OrderStatus enum as a string in the database, then we need to do the following configuration using EF Fluent API:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>()
        .Property(o => o.Status)
        .HasConversion<string>();
}
```

Code Explanations:

- **HasConversion<string>():** Converts the OrderStatus enum to its string representation when storing it in the database and converts it back when reading.

Configuring Concurrency Tokens:

Concurrency tokens help detect and handle conflicts when multiple users attempt to update the same record simultaneously. EF Core uses these tokens to implement optimistic concurrency control, preventing data overwrites. EF Core uses a Timestamp property to handle this. For example, to configure the RowVersion property as a concurrency token:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>()
        .Property(o => o.RowVersion)
        .IsRowVersion();
}
```

Code Explanation:

- **IsRowVersion():** Marks the RowVersion property as a concurrency token. EF Core uses this property to detect concurrent updates.
- **Underlying Type:** Typically, RowVersion is a byte[] that the database automatically updates on each modification.

Alternative Concurrency Tokens:

Besides row versions, you can use other properties (e.g., LastModified) as concurrency tokens. In this case, we need to use the IsConcurrencyToken method as follows:

```
modelBuilder.Entity<Order>()
    .Property(o => o.LastModified)
    .IsConcurrencyToken();
```

Configuring Shadow Properties

Shadow properties are properties that are not defined in the entity class but are part of the EF Core model and mapped to database columns. They are useful for tracking metadata, audit information, or any additional data that should be stored in the database without being part of the domain model. For example, we might want to track CreatedDate and ModifiedDate for entities but not define them in the entity class. We can configure shadow properties like this:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
```

```

modelBuilder.Entity<Product>()
    .Property<DateTime>("CreatedDate")
    .HasDefaultValueSql("GETDATE()");
}

```

Code Explanation:

- **Property<DateTime>("CreatedDate"):** Defines a shadow property named CreatedDate of type DateTime.
- **HasDefaultValueSql("GETDATE()"):** Sets the default value of CreatedDate to the current date and time when a new record is inserted.

Configuring Value Generation (Identity)

Value generation strategies define how the database generates values for certain properties. Common strategies include identity columns (auto-incremented values). Configuring these ensures that primary keys or other unique identifiers are generated appropriately. For example, to configure the ID as an identity column in the Customer entity, we need to configure the same as follows:

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .Property(c => c.Id)
        .ValueGeneratedOnAdd();
}

```

Code Explanation:

- **ValueGeneratedOnAdd():** Configures the Id property to generate its value automatically when a new record is inserted. This is typically used for identity columns.
- **Database Behavior:** For SQL Server, this translates to the IDENTITY property, which auto-increments the value.

Ignoring Properties

There are instances where certain properties of an entity should not be mapped to the database. This could be because they are used only for business logic, are computed in the application, or should remain private. EF Core allows us to exclude these properties from the model. Suppose the Product entity has a property called FullDescription that we don't want to persist in the database:

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Product>()
        .Ignore(p => p.FullDescription);
}

```

Code Explanation:

- **Ignore(p => p.FullDescription):** Excludes the FullDescription property from the EF Core model, ensuring it is not persisted to the database.
- **Use Cases:** Temporary fields, properties used solely in the application layer, or redundant data that is derived from other properties.

Complete Example with Models, DbContext, and Program Class

Let's implement a complete example to understand how these Property Configurations work together in a .NET console application using Entity Framework Core Fluent API. We will define a few models, apply property configurations, and show the output.

Creating Models

We will create entities: Customer, Product, Order, OrderItem and OrderStatus Enum. We will focus on property configurations in these entities.

OrderStatus Enum:

Create a class file named **OrderStatus.cs** within the Entities folder and copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public enum OrderStatus
    {
        Pending,
        Processing,
        Completed,
        Cancelled
    }
}
```

Customer Entity:

Create a class file named **Customer.cs** within the Entities folder and copy and paste the following code.

```
namespace EFCorePropertyConfigurations.Entities
{
    public class Customer
    {
        public int Id { get; set; } // Will configure it as Identity
        public string FirstName { get; set; } // Will configure max length
        public string LastName { get; set; }
        public string Email { get; set; } // Will configure as required
        public string PhoneNumber { get; set; } // We will make it Optional
        public DateTime? LastLoginDate { get; set; } // Nullable property
        public DateTime CreatedDate { get; set; } // Will configure default
        value public ICollection<Order> Orders { get; set; }
    }
}
```

Product Entity:

Create a class file named **Product.cs** within the Entities folder and copy and paste the following code.

```

namespace EFCorePropertyConfigurations.Entities
{
    public class Product
    {
        public int ProductId { get; set; }
        public string Name { get; set; } // Will configure column name
        public decimal Price { get; set; } // Will configure precision and
scale
        public string Description { get; set; } //Will Ignore this property
        public byte[] RowVersion { get; set; } // Will configure as
concurrency token
        public ICollection<OrderItem> OrderItems { get; set; }
    }
}

```

Order Entity:

Create a class file named **Order.cs** within the Entities folder and copy and paste the following code.

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCorePropertyConfigurations.Entities
{
    public class Order
    {
        public int OrderId { get; set; }
        public DateTime OrderDate { get; set; } // Required
        public OrderStatus Status { get; set; } // Will configure enum
mapping and also default value as Pending
        public DateTime CreatedDate { get; set; } // Will configure default
value
        public byte[] RowVersion { get; set; } // Concurrency Token
        public int CustomerId { get; set; } // Foreign Key
        public Customer Customer { get; set; }
        public ICollection<OrderItem> OrderItems { get; set; }
    }
}

```

OrderItem Entity:

Create a class file named **OrderItem.cs** within the Entities folder and copy and paste the following code.

```

namespace EFCorePropertyConfigurations.Entities
{
    public class OrderItem
    {
        public int OrderItemId { get; set; }
        public decimal UnitPrice { get; set; } // Will configure precision
and scale
        public int Quantity { get; set; }
    }
}

```

```

        public decimal TotalPrice { get; set; } // Will configure computed
column
        public int OrderId { get; set; }
        public Order Order { get; set; }
        public int ProductId { get; set; }
        public Product Product { get; set; }
    }
}

```

DbContext Class with Property Configurations

Next, we need to configure the property configurations by overriding the OnModelCreating method of the DbContext class. Modify the EFCoreDbContext class as follows. The following example code is self-explained, so please read the comment lines for a better understanding.

```

using EFCorePropertyConfigurations.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=OrderDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Configuring Customer entity
            modelBuilder.Entity<Customer>(entity =>
            {
                // Configuring column name
                entity.Property(c => c.FirstName)
                    .HasColumnName("First_Name");
                // Configuring maximum length
                entity.Property(c => c.FirstName)
                    .HasMaxLength(100);
                // Configuring required property
                entity.Property(c => c.Email)
                    .IsRequired();
                // Configuring default value
                entity.Property(c => c.CreatedDate)
                    .HasDefaultValueSql("GETDATE()");
                //Configuring Identity
                entity.Property(c => c.Id)
                    .ValueGeneratedOnAdd();
                //Configuring Nullable Property
                entity.Property(p => p.PhoneNumber)
                    .IsRequired(false);
            });
        }
    }
}

```

```

// Configuring Product entity
modelBuilder.Entity<Product>(entity =>
{
    // Configuring column name
    entity.Property(p => p.Name).HasColumnName("ProductName");
    //Configuring Column data type
    entity.Property(p => p.Price)
        .HasColumnType("decimal(18,2)");
    //Ignoring the Description Property
    entity.Ignore(p => p.Description);
    // Configuring concurrency token
    entity.Property(p => p.RowVersion)
        .IsRowVersion();
    // Configuring Shadow Property
    entity.Property<DateTime>("CreateDate")
        .HasDefaultValueSql("GETDATE()");
});
// Configuring Order entity
modelBuilder.Entity<Order>(entity =>
{
    // Configuring enum mapping to string, i.e., value conversion
    entity.Property(o => o.Status)
        .HasConversion<string>();
    // Configuring default value for CreateDate
    entity.Property(o => o.CreateDate)
        .HasDefaultValueSql("GETDATE()");
    // Configuring default value for Status
    entity.Property(o => o.Status)
        .HasDefaultValue(OrderStatus.Pending);
    // Configuring concurrency token
    entity.Property(o => o.RowVersion)
        .IsRowVersion();
});
// Configuring OrderItem entity
modelBuilder.Entity<OrderItem>(entity =>
{
    // Configuring precision and scale
    entity.Property(oi => oi.UnitPrice)
        .HasPrecision(18, 2);
    // Configuring precision and scale
    entity.Property(oi => oi.TotalPrice)
        .HasPrecision(18, 2);
    // Configuring computed column
    entity.Property(oi => oi.TotalPrice)
        .HasComputedColumnSql("[UnitPrice] * [Quantity]");
});
}
// Define DbSets
public DbSet<Customer> Customers { get; set; }
public DbSet<Product> Products { get; set; }
public DbSet<Order> Orders { get; set; }

```

```

    public DbSet<OrderItem> OrderItems { get; set; }
}
}

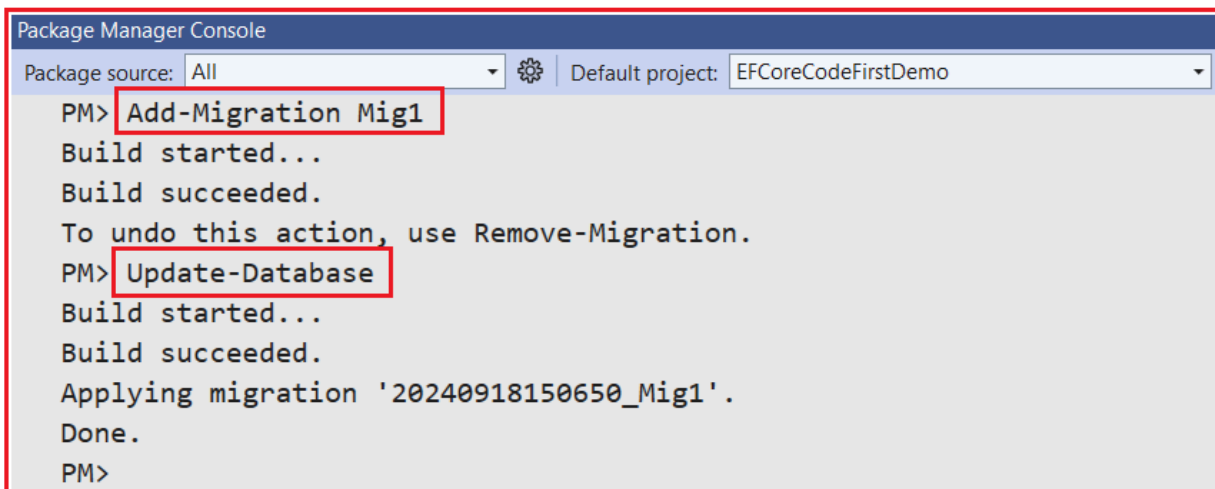
```

Key Points:

- **Column Names:** The FirstName property of the Customer entity is mapped to the column First_Name. The Name property of the Product entity is mapped to the column ProductName.
- **Data Types:** The Price property of the Product entity and the Total Price and Unit Price Properties of the Order Item Entity is configured with a precision of 18 and a scale of 2 (decimal(18,2)).
- **Default Values:** The CreatedDate properties of both the Customer, Product, and Order entities have default values of GETDATE() using HasDefaultValueSql("GETDATE()"). Additionally, a shadow property CreatedDate for Product is also configured with a default value of GETDATE().
- **Nullable and Required:** The Email property of the Customer entity is required using IsRequired(). The PhoneNumber property is configured as an optional (nullable) property using IsRequired(false).
- **Maximum Length:** The FirstName property of the Customer entity has a maximum length of 100 characters, configured using HasMaxLength(100).
- **Computed Columns:** The TotalPrice property of the OrderItem entity is computed in the database using the SQL expression [UnitPrice] * [Quantity], configured with HasComputedColumnSql().
- **Value Conversions:** The Status property of the Order entity (an enum) is stored as a string using HasConversion<string>().
- **Concurrency Tokens:** The RowVersion property of the Product and Order entities is configured as a concurrency token using IsRowVersion().
- **Ignored Properties:** The Description property of the Product entity is ignored and not mapped to the database.

Generating the Migration and Updating the Database

Open the Package Manager Console and execute the Add-Migration and Update-Database commands as shown in the below image:

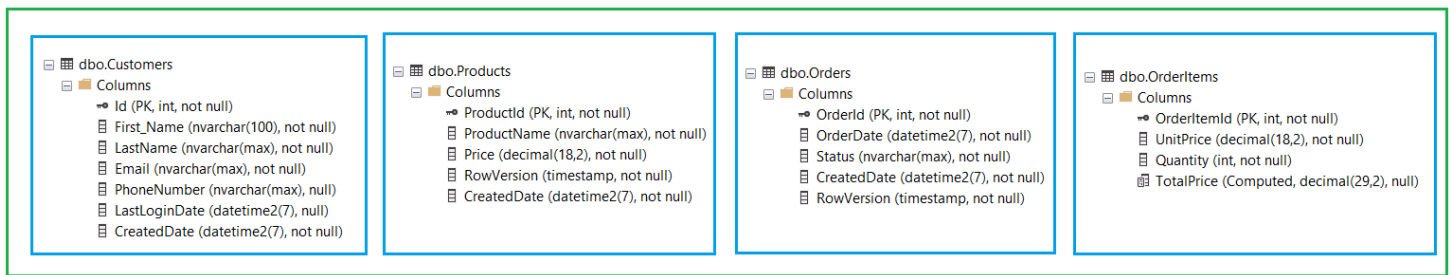


```

Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240918150650_Mig1'.
Done.
PM>

```

Once you execute the above command, it will create the OrderDB database with the configured entities and properties, as shown in the image below.



Testing the Functionalities

Now, we will develop one application to test the following functionalities:

- Creating Products
- Creating Customers
- Creating Orders with OrderItems
- Displaying Orders

So, modify the Program Class as follows:

```
using EFCorePropertyConfigurations.Entities;
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            // Initialize the database context
            using (var context = new EFCoreDbContext())
            {
                // Step 1: Create and add some products
                CreateProducts(context);
                // Step 2: Create and add some customers
                CreateCustomers(context);
                // Step 3: Create and add some orders with related order
                items CreateOrdersWithOrderItems(context);
                // Step 4: Display all customers and their respective orders
                DisplayAllOrders(context);
            }
        }
        // Step 1: Create and save a list of products to the database
        private static void CreateProducts(EFCoreDbContext context)
        {
            // Check if products already exist to avoid duplication
            if (context.Products.Any())
                return;
            // List of products to add
            var products = new List<Product>
```

```

    {
        new Product { Name = "Laptop", Price = 1200m },
        new Product { Name = "Smartphone", Price = 800m },
        new Product { Name = "Headphones", Price = 150m },
        new Product { Name = "Tablet", Price = 500m }
    };
    // Add products to the context
    context.Products.AddRange(products);
    // Save changes to the database
    context.SaveChanges();
    Console.WriteLine("Products have been added to the database.");
}
// Step 2: Create and save a list of customers to the database
private static void CreateCustomers(EFCoreDbContext context)
{
    // Check if customers already exist to avoid duplication
    if (context.Customers.Any())
        return;
    // List of customers to add
    var customers = new List<Customer>
    {
        new Customer
        {
            FirstName = "Pranaya",
            LastName = "Rout",
            Email = "Pranaya.Rout@dotnettutorials.net",
            PhoneNumber = "123-456-7890"
        },
        new Customer
        {
            FirstName = "Rakesh",
            LastName = "Kumar",
            Email = "Rakesh.Kumar@dotnettutorials.net",
            PhoneNumber = "098-765-4321"
        }
    };
    // Add customers to the context
    context.Customers.AddRange(customers);
    // Save changes to the database
    context.SaveChanges();
    Console.WriteLine("Customers have been added to the database.");
}
// Step 3: Create and save orders with related order items
private static void CreateOrdersWithOrderItems(EFCoreDbContext
context)
{
    // Check if orders already exist to avoid duplication
    if (context.Orders.Any())
        return;
    // Fetch the customers who are going to place orders

```

```

        var customer1 = context.Customers.First(c => c.Email ==
"Pranaya.Rout@dotnettutorials.net");
        var customer2 = context.Customers.First(c => c.Email ==
"Rakesh.Kumar@dotnettutorials.net");
        // Fetch existing products
        var laptop = context.Products.First(p => p.Name == "Laptop");
        var smartphone = context.Products.First(p => p.Name ==
"Smartphone");
        // Create orders for each customer
        var orders = new List<Order>
        {
            new Order
            {
                CustomerId = customer1.Id,
                OrderDate = DateTime.Now,
                //Status = OrderStatus.Pending, By Default it should be
Pending
                OrderItems = new List<OrderItem>
                {
                    new OrderItem
                    {
                        ProductId = laptop.ProductId,
                        UnitPrice = laptop.Price,
                        Quantity = 1
                    },
                    new OrderItem
                    {
                        ProductId = smartphone.ProductId,
                        UnitPrice = smartphone.Price,
                        Quantity = 2
                    }
                }
            },
            new Order
            {
                CustomerId = customer2.Id,
                OrderDate = DateTime.Now,
                Status = OrderStatus.Completed,
                OrderItems = new List<OrderItem>
                {
                    new OrderItem
                    {
                        ProductId = smartphone.ProductId,
                        UnitPrice = smartphone.Price,
                        Quantity = 2
                    }
                }
            }
        };
        // Add orders to the context
        context.Orders.AddRange(orders);

```

```

        // Save changes to the database
        context.SaveChanges();
        Console.WriteLine("Orders have been created for the customers.");
    }
    // Step 4: Display all customers and their orders, including order
items
    private static void DisplayAllOrders(EFCoreDbContext context)
    {
        // Fetch all customers with their orders and order items using
eager loading
        var customersWithOrders = context.Customers
            .Include(c => c.Orders) // Eager load related orders
            .ThenInclude(o => o.OrderItems) // Eager load related
order items
            .ThenInclude(oi => oi.Product) //Eager Load Related
Products of OrderItems
            .ToList();
        Console.WriteLine("\n----- All Orders for All
Customers ----- \n");
        // Loop through each customer
        foreach (var customer in customersWithOrders)
        {
            Console.WriteLine($"Customer: {customer.FirstName}
{customer.LastName} ({customer.Email})");
            // Loop through each order of the customer
            foreach (var order in customer.Orders)
            {
                Console.WriteLine($"Order ID: {order.OrderId}, Order
Status: {order.Status}, Order Date: {order.OrderDate:yyyy-MM-dd}");
                Console.WriteLine("Order Items:");
                // Loop through each order item
                foreach (var item in order.OrderItems)
                {
                    Console.WriteLine($" \tProduct Name: Name:
{item.Product.Name}, Unit Price: {item.UnitPrice}, Quantity:
{item.Quantity}");
                    Console.WriteLine($" \tTotal Price for Item:
{item.TotalPrice}");
                }
                // Calculate total order cost
                var totalOrderCost = order.OrderItems.Sum(oi =>
oi.TotalPrice);
                Console.WriteLine($"Total Order Cost:
{totalOrderCost}\n");
            }
        }
    }
}

```

Key Points for Understanding:

- **Creating Products:** This section adds predefined products (Laptop, Smartphone, etc.) into the database using `context.Products.AddRange()` and save the changes using `context.SaveChanges()`.
- **Creating Customers:** Two customers are created with basic information such as first name, last name, email, and phone number. These are added and saved to the database in a similar manner to the products.
- **Creating Orders with Related Order Items:** Orders are created for existing customers, and each order is associated with items that reference previously created products (laptop and smartphone). These orders are then saved to the database.
- **Displaying All Customers and Orders:** Eager loading (`Include`) is used to fetch all customers along with their related orders, order items and product information. The output displays the customer's full name, order details (ID, status, date), and order items (unit price, quantity, total price for each item, and total order cost) and product name.

So, when you run the above application, the output will display a list of customers, their orders, and the related order items with detailed pricing information, as shown in the image below:

```
Products have been added to the database.
Customers have been added to the database.
Orders have been created for the customers.

----- All Orders for All Customers -----

Customer: Pranaya Rout (Pranaya.Rout@dotnettutorials.net)
Order ID: 1, Order Status: Pending, Order Date: 2024-09-18
Order Items:
    Product Name: Name: Laptop, Unit Price: 1200, Quantity: 1
    Total Price for Item: 1200
    Product Name: Name: Smartphone, Unit Price: 800, Quantity: 2
    Total Price for Item: 1600
Total Order Cost: 2800

Customer: Rakesh Kumar (Rakesh.Kumar@dotnettutorials.net)
Order ID: 2, Order Status: Completed, Order Date: 2024-09-18
Order Items:
    Product Name: Name: Smartphone, Unit Price: 800, Quantity: 2
    Total Price for Item: 1600
Total Order Cost: 1600
```

Benefits of Property Configurations in EF Core

- **Full Control:** Fluent API provides fully control over how each property is mapped to the database.
- **Data Integrity:** Ensures that properties have correct data types, lengths, and constraints, enhancing data integrity.
- **Database Performance:** Properly configuring data types and constraints can improve query performance.
- **Business Logic:** Ensures that business rules (such as default values and computed columns) are enforced at the database level.
- **Separation of Concerns:** Keeps entity classes clean by separating configuration from business logic.

These are Entity Framework Core's most commonly used property configurations using Fluent API. Property configurations allow us to have more control over how properties are represented and managed in the underlying database, ultimately improving data integrity and consistency.

In the next article, I will discuss [How to Create a table without a Primary Key in Entity Framework Core using Fluent API](#) with Examples. In this article, I explain **Property Configuration in Entity Framework Core using Fluent API** with examples. I hope you enjoyed this Property Configuration in Entity Framework Core using Fluent API article.